



- Managed Service Provider
- DevOps Services Competency
- Migration Services Competency
- Well-Architected Partner Program

- Digital Workplace Services Competency
- AWS Microsoft Workloads Services Competency



Wonder Home Finance

Cross-Region DR Migration – MySQL Workloads

Region: Mumbai → Hyderabad

July 27, 2026

Plan of Action (POA)

Cross-Region DR Migration – MySQL Workloads (Mumbai → Hyderabad)

Field	Detail
Prepared By	Bikram – Data Engineer, Rapyder Cloud Solutions
Document Type	Plan of Action (POA)
Region Pair	ap-south-1 (Mumbai) → ap-south-2 (Hyderabad)
Scope	Migration 1: Self-managed MySQL on EC2 Migration 2: Amazon RDS for MySQL
Classification	Client-Facing – Technical

1. Objective

This document defines the plan of action for establishing disaster-recovery (DR) capable MySQL environments in Hyderabad (ap-south-2), replicated in near real-time from the existing production environments in Mumbai (ap-south-1). Two independent migration/replication tracks are covered, since the source engines differ in deployment model:

- Migration 1 – Self-managed MySQL on EC2 in Mumbai, replicated to self-managed MySQL on EC2 in Hyderabad using native MySQL replication (binlog-based).
- Migration 2 – Amazon RDS for MySQL in Mumbai, replicated to Amazon RDS for MySQL in Hyderabad using an AWS cross-region read replica.

Both tracks are designed to run in parallel with independent cutover/failover procedures, since the underlying replication mechanisms and operational controls differ.

2. Current State – Assumptions

- Source EC2 MySQL instance is reachable, has binary logging enabled or can be enabled with a restart during a maintenance window.
- Source RDS MySQL instance has automated backups enabled (a prerequisite for cross-region read replicas).
- VPC peering / Transit Gateway connectivity between Mumbai and Hyderabad VPCs is either already in place or will be provisioned as part of this POA.
- Security groups, NACLs, and route tables will be updated to permit MySQL (3306) traffic between the two regions.
- Target region (Hyderabad) has equivalent subnet groups, parameter groups, and option groups (for RDS) pre-created or planned.

3. Migration 1: EC2 MySQL → EC2 MySQL (Native Replication)

3.1 Approach

Use native asynchronous MySQL replication (binlog-based) with the Hyderabad EC2 instance configured as a replica of the Mumbai EC2 source. GTID-based replication is preferred over log-position-based replication for resilience during failover.

3.2 Pre-requisites

- Provision target EC2 instance in Hyderabad matching source instance class, storage type/IOPS, and MySQL version exactly.
- Enable binary logging on source (log_bin, server_id, binlog_format=ROW) if not already enabled — requires a restart, to be scheduled in a maintenance window.
- Create a dedicated replication user on source, scoped to REPLICATION SLAVE / REPLICATION CLIENT privileges only.
- Open port 3306 between Hyderabad EC2 security group and Mumbai EC2 security group over the inter-region network path.

3.3 Execution Steps

1. Take a consistent snapshot of the source dataset using mysqldump or Percona XtraBackup for larger datasets, capturing the binlog position/GTID at snapshot time.
2. Provision and patch the target EC2 instance in Hyderabad with the same MySQL version, then restore the snapshot.
3. Configure the target as a replica: CHANGE MASTER TO (or CHANGE REPLICATION SOURCE TO on MySQL 8.0.23+) pointing to the Mumbai source, using the replication user and either GTID auto-position or the captured binlog coordinates.
4. Start replication (START REPLICATION / START SLAVE) and confirm via SHOW REPLICATION STATUS that Replica_IO_Running and Replica_SQL_Running are both Yes.
5. Monitor Seconds_Behind_Source until it reaches 0, confirming the replica has caught up to the source.
6. Validate row counts and checksums (pt-table-checksum or CHECKSUM TABLE) on a sample of critical tables between source and target.
7. Configure alerting on replication lag and replica thread status (CloudWatch agent or a cron-based check calling SHOW REPLICATION STATUS).
8. Document the failover procedure: promote Hyderabad replica (STOP REPLICATION, RESET REPLICATION ALL), re-point application connection strings/DNS, and re-establish reverse replication once Mumbai is available again.

3.4 Validation & Sign-off Criteria

- Replication lag stable at 0–2 seconds under normal load for 48 hours.
- Checksum validation shows no data drift on sampled tables.

4. Migration 2: RDS MySQL → RDS MySQL (Cross-Region Read Replica)

4.1 Approach

Use AWS-managed cross-region read replication for Amazon RDS MySQL. AWS handles the underlying binlog shipping between regions; the operational surface is the RDS/Console/CLI layer rather than raw MySQL replication commands.

4.2 Pre-requisites

- Confirm automated backups are enabled on the Mumbai source RDS instance (mandatory for creating a cross-region read replica).
- Confirm source engine version is supported for cross-region replicas; align parameter group settings (binlog_format=ROW, backup retention) between regions.
- Create a DB subnet group in the Hyderabad VPC covering at least two AZs.
- Ensure KMS key strategy is defined if the source is encrypted — cross-region replicas require a KMS key in the destination region, with a re-encryption step during replica creation.
- Confirm VPC connectivity (peering/TGW) and security group rules allow the cross-region replication traffic AWS manages internally, plus any application-side connectivity to the new endpoint.

4.3 Execution Steps

1. From the RDS console/CLI, initiate 'Create Read Replica' on the Mumbai source instance, selecting Hyderabad (ap-south-2) as the destination region.
2. Specify target instance class, storage, Multi-AZ setting, subnet group, parameter group, and (if source is encrypted) the destination-region KMS key.
3. Allow AWS to provision the replica and begin seeding — monitor via the Replica Lag and BinLogDiskUsage CloudWatch metrics on the source.
4. Once initial seeding completes, monitor ReplicaLag on the Hyderabad replica until it stabilizes near 0.
5. Validate connectivity from application/test hosts in Hyderabad to the new replica endpoint.
6. Run data validation checks (row counts, checksums on key tables) comparing source and replica.
7. Configure CloudWatch alarms on ReplicaLag and replica storage/CPU utilization.

8. Document the promotion procedure: 'Promote Read Replica' action (breaks replication, makes the Hyderabad instance standalone and writable), followed by DNS/connection-string cutover and, if required, re-establishing replication back to Mumbai post-recovery.

4.4 Validation & Sign-off Criteria

- ReplicaLag consistently under 5 seconds under normal production load for 48 hours.
- Backup and monitoring configuration confirmed on the promoted instance profile.

5. Risks & Mitigations

Risk	Impact	Mitigation
Binlog not enabled on EC2 source	Requires restart to enable, causing brief downtime	Schedule during an approved maintenance window; validate binlog_format beforehand
Cross-region network latency affecting replication lag	Elevated ReplicaLag / Seconds_Behind_Source	Right-size instance classes; monitor lag continuously; consider parallel replication threads on EC2 side
KMS key mismatch during RDS replica creation	Replica creation failure	Pre-create and validate destination KMS key before initiating replica creation
Schema drift between regions over time	Replication breaks on DDL	Apply all schema changes through a single controlled pipeline; monitor replication error logs

6. Timelines

Migration 1: Mumbai EC2 MySQL to Hyderabad EC2 MySQL (Native MySQL Replication)

Day	Activity	Description	Effort
Day 1	Environment Assessment	Review source and target EC2 instances, MySQL versions, storage, networking, firewall rules, and replication prerequisites.	2 hrs
	Hyderabad EC2 Setup	Provision/configure EC2, install MySQL, configure storage, users, and required directories.	3 hrs
	MySQL Configuration	Configure server_id, binary logging, GTID (if applicable), replication user, and restart MySQL services.	2 hrs
	Connectivity Validation	Verify EC2-to-EC2 connectivity over port 3306 and security group/firewall configuration.	1 hr
	Total		8 hrs
Day 2	Initial Data Synchronization	Take backup from Mumbai EC2 and restore to Hyderabad EC2.	6 hrs
	Configure Native Replication	Configure replication using binary log coordinates or GTID and start replication.	2 hrs
	Total		8 hrs

Day 3	Replication Validation	Verify IO/SQL threads, replication status, and data consistency.	2 hrs
	Documentation	Document replication configuration and validation results.	1 hr
	Buffer / Final Validation	Final testing, issue resolution, and sign-off.	2 hrs
	Total		5 hrs

Migration 2: Mumbai Amazon RDS MySQL to Hyderabad Amazon RDS MySQL (AWS Cross-Region Read Replica)

Day	Activity	Description	Effort
Day 1	Prerequisite Validation	Verify automated backups, parameter groups, networking, and IAM permissions.	2 hrs
	Create Cross-Region Read Replica	Create read replica in Hyderabad using the AWS Console.	2 hrs
	Replica Monitoring	Monitor replica creation and replication health.	2 hrs
	Connectivity Validation	Verify connectivity from application servers and validate replica accessibility.	2 hrs
	Total		8 hrs
Day 2	Replication Validation	Verify replication lag, data consistency, and database health.	2 hrs
	DR Documentation	Document replication setup and validation results.	2 hrs
	Final Verification & Sign-off	Final health checks, monitoring verification, and handover.	4 hrs
	Total		8 hrs